Summary (slides)

Polymorphism
General Concept
Etymology: Derived from Greek, meaning "having many forms"

The ability to treat related objects (such as different chess pieces, farm animals, or vehicles) uniformly within client code, while ensuring that runtime, each individual object automatically executes the correct behavior corresponding to its actual type

Shorter, cleaner client code and massive improvements is extensibility - new behaviors or classes can be added seamlessly without altering the core client logic

Cons/Trade-offs: You sacrifice compile-time certainty, exact execution behavior isn't determined until runtime based on the actual object instance.

Abstraction

Abstraction is the core process of distilling a software component or concept by highlighting only the information relevant to a specific context while suppressing unnecessary details. In object-oriented programming, this is achieved through abstract classes and interfaces - which define capabilities without revealing internal implementation details - and is governed by five strict adherence criteria:

- using precise
- meaningful names (nouns for classes, verbs, methods maintaining context awareness, providing an explicit declaration of the public interface, ensuring by avoiding unnecessary redundancies. A well-designed abstraction prevents common pitfalls like "leaky abstractions" or "over-abstraction", dramatically

improving software modularity, readability and maintainability.

Interface Design

User Interface (UI) design fundamentally begins with a deep, human-centric understanding of people and usage patterns rather than technical constraints.

Developers must prioritize observing actual behavior during requirements elicitation over merely listening to users, utilizing personas, models an scenarios and prototypes to bridge the gap between human needs and software implementation.

A major challenge in this domain is managing system complexity. Software naturally accumulates inherent domain complexity alongside accidental, self-inflicted technical complications, yet a complex backend should never dictate a convoluted user interface. Guided by Jef Raskin's four laws of a humane interface, software must protect user work, eliminate wasted time and unnecessary effort, respond empathetically to human needs and frailties and allow users to control the exact pace of interaction. Ultimately, successful design is driven by user-centered, human-centered, and usage-centered methodologies, steering completely clear of ineffective, developer-centric approaches driven solely by underlying databases, data structures, programming languages or frameworks.